

For a packet sniffer like this one, the program may have to process thousands of packets in a session where multiple nodes are simultaneously transmitting and receiving packets in a network. This generates lots of traffic on the network as time is needed to transmit, receive, process the packet, wait times for various nodes to receive signals that can continue processing. With the use of a single thread, only one process can be executed at a time leaving the thousands of packets to have to wait in queue until it is its turn to be processed so that it can finally arrive at its destination. However with multi-threading, each thread can be assigned to process an individual packet each and work in parallel allowing for multiple packets to be processed at a given time. This majorly increases efficiency as the total processing time decreases in non-linear time (total no. packets / no. threads) vs (total no. packets / 1) ultimately resulting in less congestion on a network and increasing throughput so that users can expect lower wait times to receive resources they may request on the Internet.

One key aspect about multithreading is the ability to work in parallel so that the completion of multiple tasks can progress at the same time however for true simultaneous work, one CPU core can have up to one thread scheduled to it. This is due to the architecture of a CPU only allowing for one instruction stream directed by a single program counter to be ran at a time therefore only one Fetch Decode Execute Cycle can be ran at a time therefore only one thread instruction can be ran at a given time. There are a number of techniques like pipelining that CPU can perform however this runs multiple threads on the single instruction stream giving the illusion of parallelism when in reality it is concurrency where the core takes turns progressing through each thread's instructions allowing for all threads to progress together. With this in mind, the program was initially implemented with 4 threads as the average CPU tends to have 4 cores. However after testing with 4,6 and 8 threads, 6 threads produced a max round trip time (RTT) of 1.4ms compared to 2.6ms with 4 threads and 1.5ms with 8 threads when 10000 SYN packets were sent. Perhaps this showed that more threads increased CPU utilisation however using too many started to incur a noticeable overhead most likely from context switching. As a result, 6 threads was chosen for the threadpool.

To put multithreading into application, the process consisted of two functions, `dispatch()` and `worker_thread*()`. `dispatch()` is called for every packet that is captured by the sniffer and adds the packet into queue for whenever a worker thread is not processing another packet and is available. Each thread as an instance of `worker_thread` running and how it works is, it constantly runs a loop which accesses the queue for the next packet to be processed and once a packet has been extracted from the queue, the thread calls `analyse()`, the main routine used to parse the contents of the packet and determine the type of network traffic or attack. By separating packet capture from processing, this

architecture maximizes throughput reducing the time packets spend waiting to be analyzed.

The main challenge when implementing multi-threading is synchronisation. This is the idea that because the threads simultaneously work on the same data (known as the critical section), updates made by one thread may cause updates from another thread to be lost. With the use of synchronisation techniques such as mutex locks, loss of data through updates can be prevented by locking the critical section and allocating the area to one thread making sure other threads do not have access until the thread currently in the critical section is finished working there.

An alternate approach could be done with semaphores which use a counter to limit how many threads can access the critical section. Threads use the functions `wait()` to check if the critical section is available (indicated by a semaphore value greater than 0) and `signal()` to indicate the thread has finished working within the critical section and that another thread can take its place.

This approach of synchronisation was however not favoured because it did not enforce strict mutual exclusivity like mutex locks did. In theory, only one thread should be allowed to access the packet queue and packet counters used later for the output (both making up the critical section in this program) as this eliminates the possibility of multiple threads working on the same packet. Generally semaphores are used to allow threads to communicate with each other which mutex locks cannot do since multiple threads cannot access the same memory together however this is not beneficial in this use case and is actually undesirable since packet data should be processed separately to prevent interference and mixing of data from other packets therefore also acting as a form of security reinforced by the modular nature of mutex locks.

Having one mutex lock variable means that once it is locked by one thread, it cannot be accessed by other threads until it is unlocked. This is fine for synchronisation and preventing data corruption however it can be very inefficient using one global mutex as for processes that execute simultaneously, they may hold the mutex while performing relatively long operations such as enqueueing a task to the queue or using global variables and having to update them live. This can result in thread starvation, where some threads are forced to wait unnecessarily long, reducing overall program throughput. To resolve this, an additional mutex variable was added in the `analysis.c` file and is primarily used to update the global variables that are shared between threads such as the packet counters and IP array keeping this separate from the queue mutex.

Since a number of dynamic data structures were implemented, the memory allocated had to be freed at the end of the execution of the program. The program was

implemented with additional functions to abstract bigger processes like packet parsing to make understanding each step easier as well as creating a more modular design where each component could be separately debugged and tested. This however complicates the call stack making it uncertain when to free memory allocated to dynamic structures and increased the risk of accidentally freeing memory too early therefore leading to errors when accessing the structures that no longer exist. For that reason, the `free()` statements were called at the last lines of each function body and were primarily placed in functions which are likely to be lower down in the call stack like the worker thread function.

The program carefully manages all dynamically allocated memory such as packet headers, payloads, and thread argument structures, by utilising pointers. Before dereferencing any pointer, null checks are performed to prevent invalid memory access or segmentation faults. For data that should never be altered, such as packet headers and payloads passed to functions, `consts` are used to declare pointers preventing accidental modification and improving code safety. After testing with `valgrind`, it was confirmed that no memory was leaked.

Once the program was implemented with the `pcap_loop`, a signal handler function had to be implemented so that the halting and freeing of threads can take place safely. The termination of execution also provided an ideal time to output the packet counters since they would no longer update and lead to the output of out of date data. The signal handler function works by being called when the signal bit changes (to 1) after `ctrl + c` are pressed. This breaks the `pcap_loop` that runs indefinitely and leads to thread operations halting.

The program was initially tested with normal working cases of each type of attack. This included standard packet sizes and the correct flag bits (SYN or ACK) being activated. Once these proved to give the correct count of packets(should have included all packets sent), further extensive testing was done. This included using invalid formats for IP and MAC addresses for SYN and ARP attacks by either adding / removing digits or adding letters in the IP address using characters outside the expected hexadecimal range for MAC addresses. For Blacklisted URLs, identical HTTP headers were used with slight variations in the host name to see whether the parsing logic and domain-matching functionality could reliably detect blacklist violations without false positives.

The program was also tested with high volumes of packets being sent upwards of 30000 to see how the threads would handle high traffic and if there was any chance of interference or data loss this proved to be okay as there were no faults or errors that caused any crashes.